Practical – 5

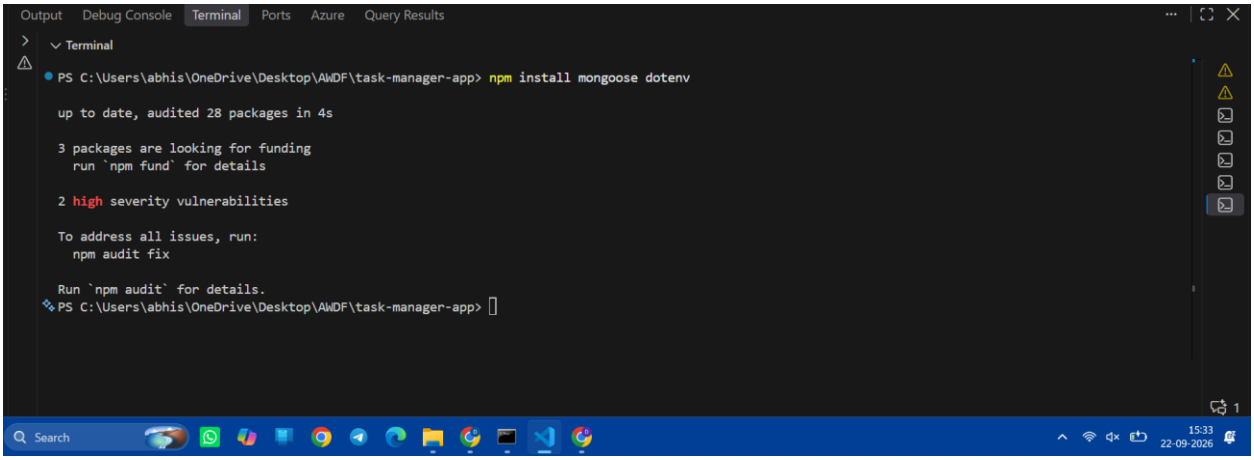
Aim:

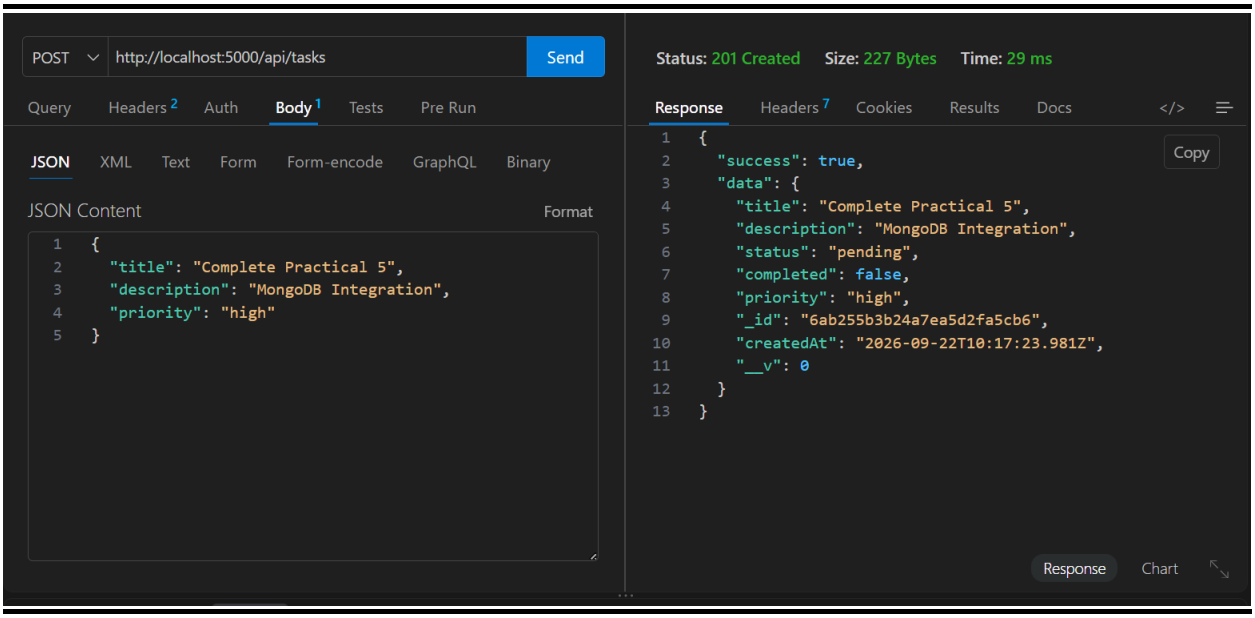
To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema

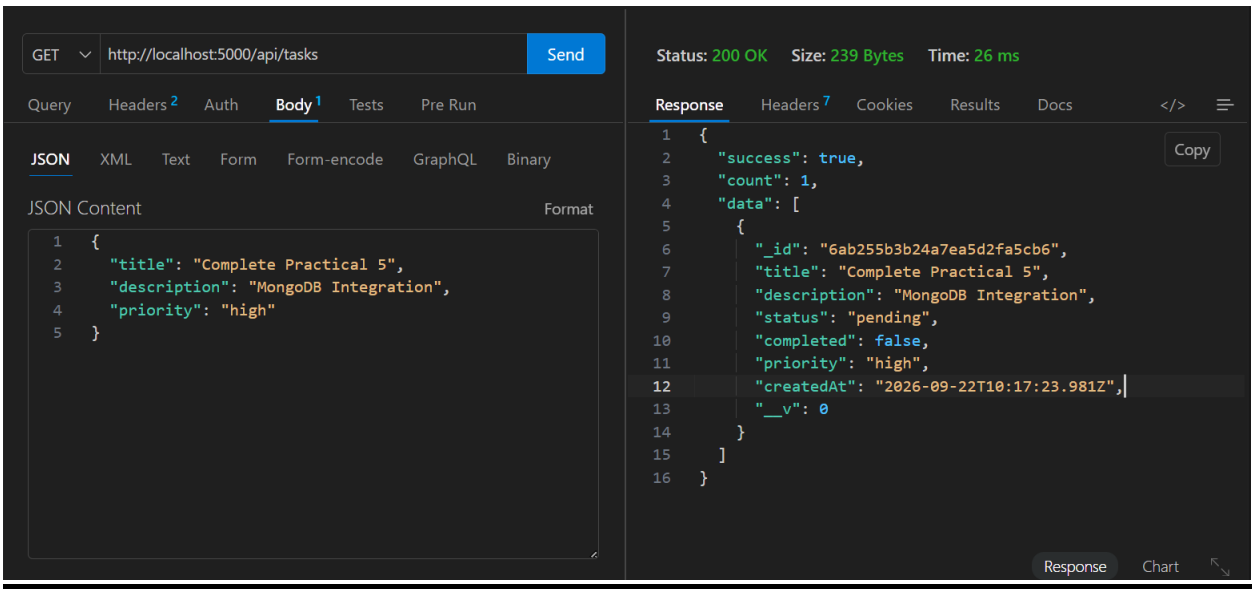
Content:

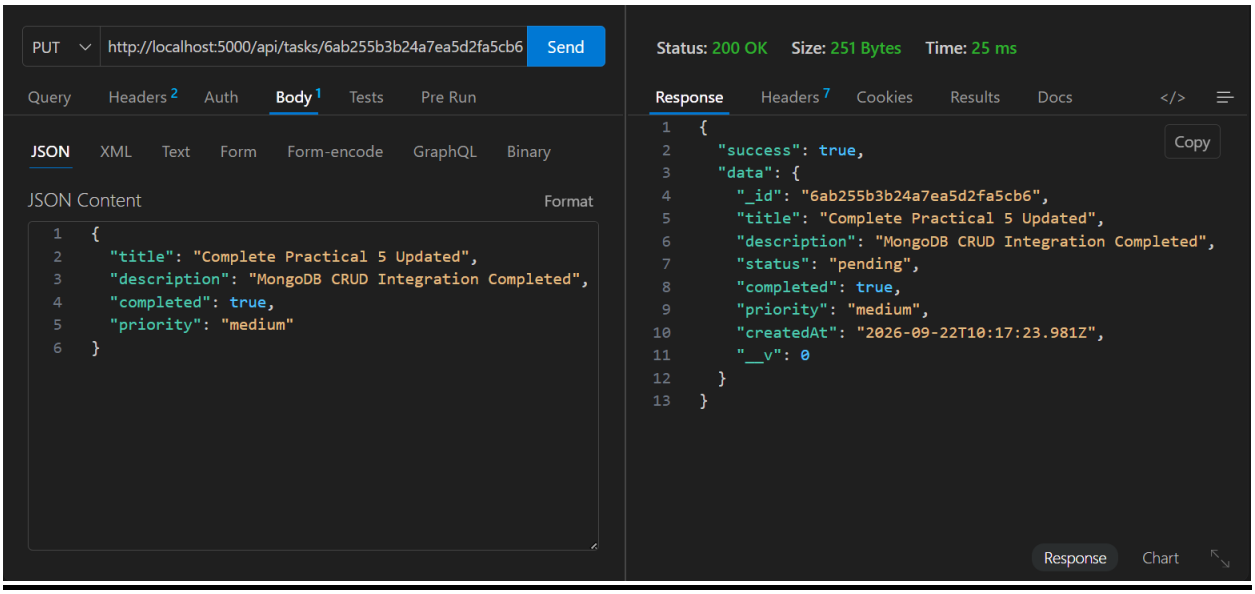
After completing this practical, I will be able to design and implement a RESTful backend server using Node.js, Express.js, MongoDB, and Mongoose. I will understand how to connect an Express application with MongoDB and create REST API endpoints for performing CRUD operations such as creating, reading, updating, and deleting tasks using HTTP methods including POST, GET, PUT, and DELETE**.** I will learn how to define a structured task schema using Mongoose with fields, default values, validation, and enumerated values. Additionally, I will understand how to handle invalid requests, validate task IDs, and provide meaningful HTTP status codes and JSON responses. I will be able to test the implemented API endpoints using tools such as Thunder Client . This practical will help me develop structured, reliable, and maintainable backend applications with Node.js, Express.js, MongoDB, and Mongoose.

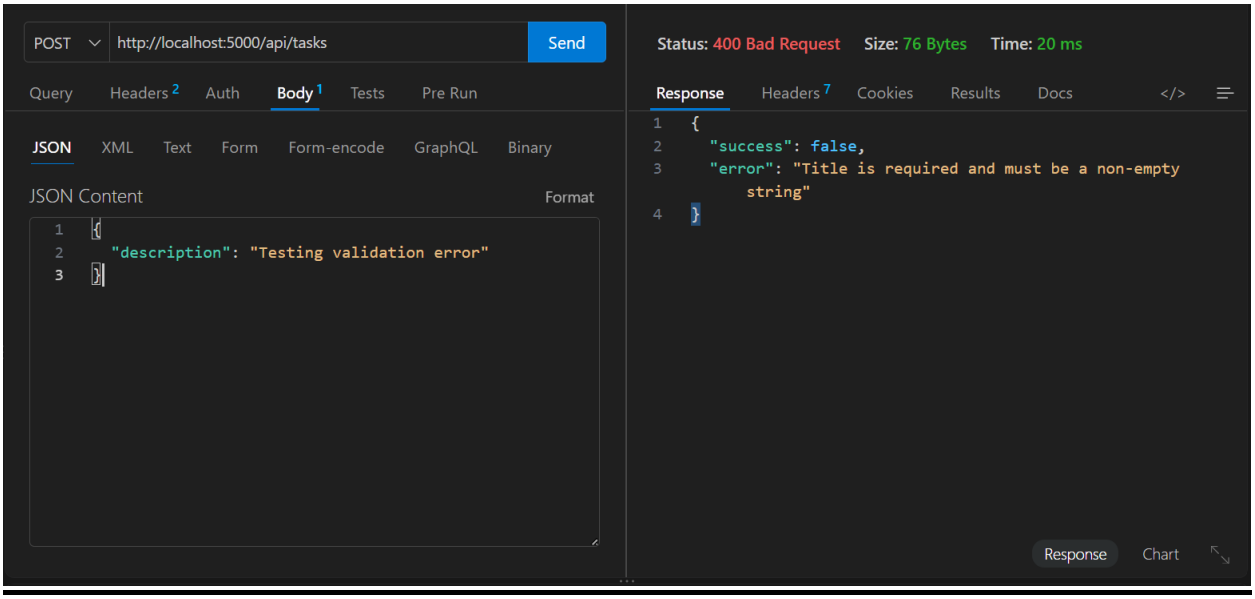
Screen Shot:

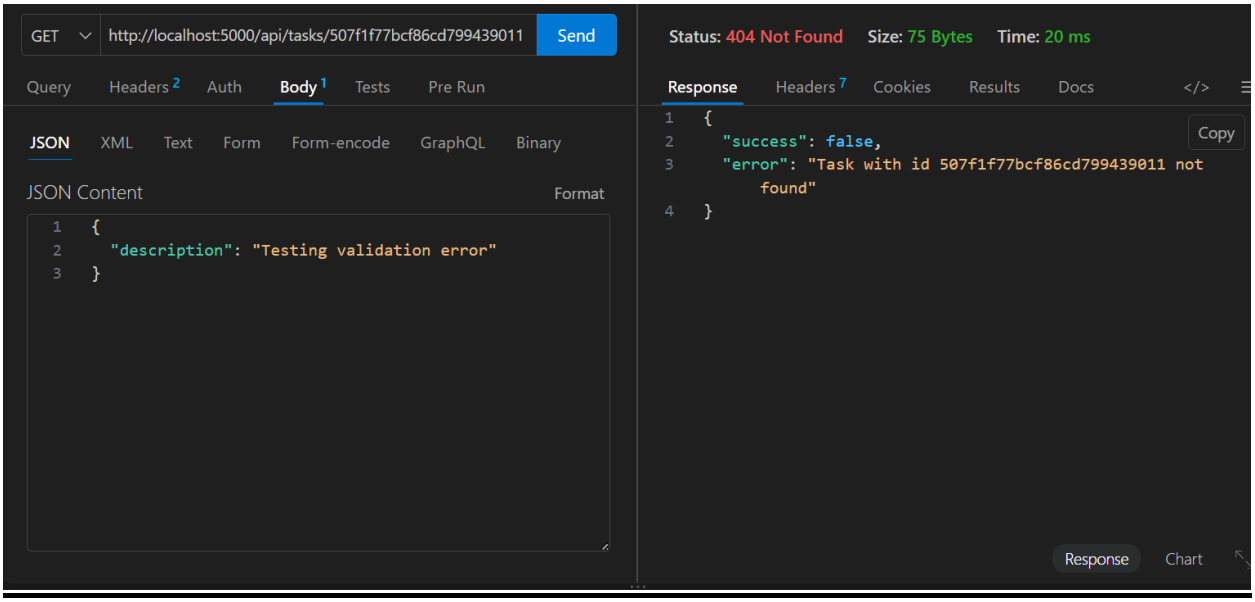

Output
Debug Console
Terminal
Ports
Azure
Query Results
Terminal
PS C:\Users\abhis\OneDrive\Desktop\AWDF\task-manager-app> npm install mongoose dotenv
up to date, audited 28 packages in 4s
3 packages are looking for funding
run `npm fund` for details
2 high severity vulnerabilities
To address all issues, run:
npm audit fix
Run `npm audit` for details.
PS C:\Users\abhis\OneDrive\Desktop\AWDF\task-manager-app>
Search
15:33
22-09-2026

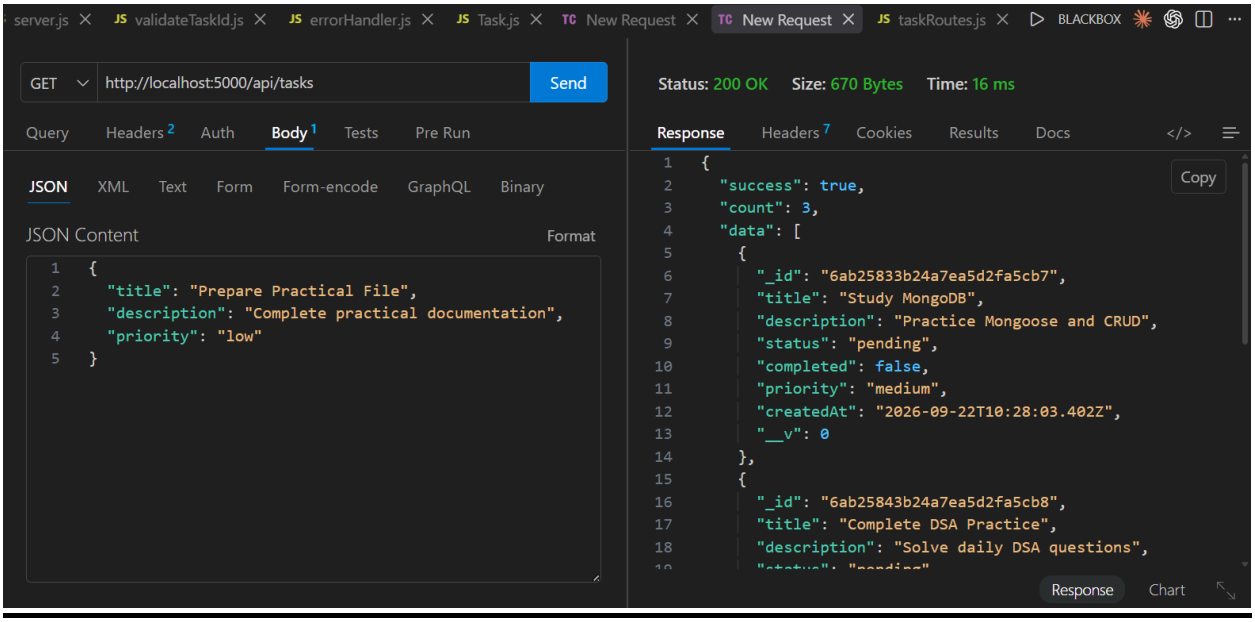

POST
http://localhost:5000/api/tasks
Send
Status: 201 Created
Size: 227 Bytes
Time: 29 ms
Query
Headers 2
Auth
Body 1
Tests
Pre Run
Response
Headers 7
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
Copy
{
"title": "Complete Practical 5",
"description": "MongoDB Integration",
"priority": "high"
}
{
"success": true,
"data": {
"title": "Complete Practical 5",
"description": "MongoDB Integration",
"status": "pending",
"completed": false,
"priority": "high",
"_id": "6ab255b3b24a7ea5d2fa5cb6",
"createdAt": "2026-09-22T10:17:23.981Z",
"__v": 0
}
}
Response
Chart

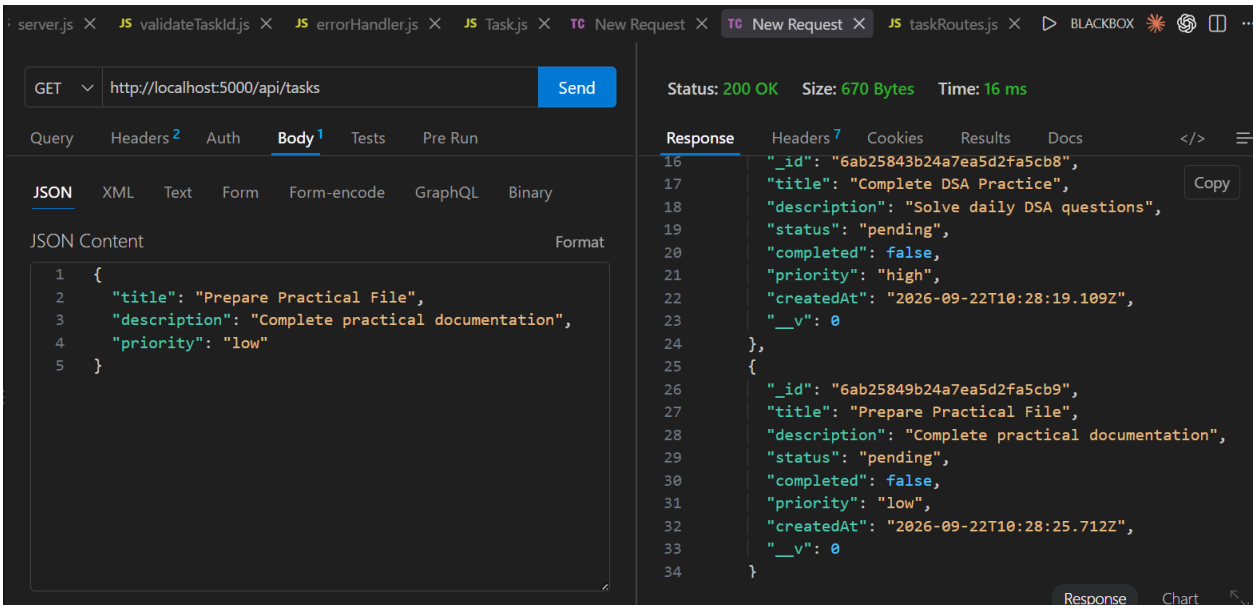

GET
http://localhost:5000/api/tasks
Send
Status: 200 OK
Size: 239 Bytes
Time: 26 ms
Query
Headers 2
Auth
Body 1
Tests
Pre Run
Response
Headers 7
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
Copy
{
"title": "Complete Practical 5",
"description": "MongoDB Integration",
"priority": "high"
}
{
"success": true,
"count": 1,
"data": [
{
"_id": "6ab255b3b24a7ea5d2fa5cb6",
"title": "Complete Practical 5",
"description": "MongoDB Integration",
"status": "pending",
"completed": false,
"priority": "high",
"createdAt": "2026-09-22T10:17:23.981Z",
"__v": 0
}
]
}
Response
Chart

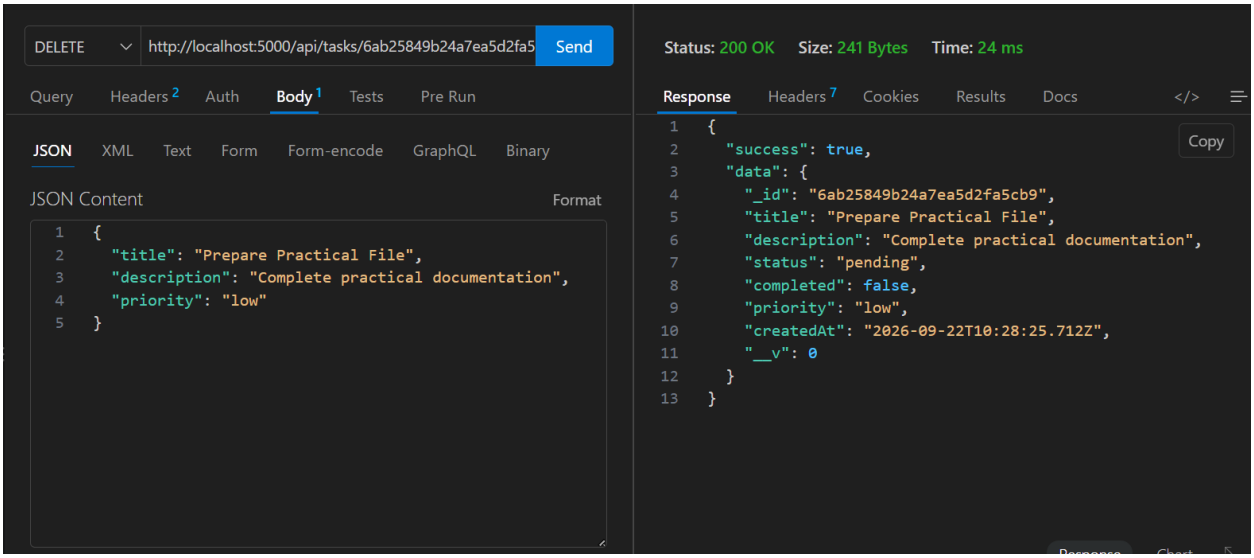

PUT
http://localhost:5000/api/tasks/6ab255b3b24a7ea5d2fa5cb6
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"title": "Complete Practical 5 Updated",
"description": "MongoDB CRUD Integration Completed",
"completed": true,
"priority": "medium"
}
Status: 200 OK
Size: 251 Bytes
Time: 25 ms
Response
Headers 7
Cookies
Results
Docs
Copy
{
"success": true,
"data": {
"_id": "6ab255b3b24a7ea5d2fa5cb6",
"title": "Complete Practical 5 Updated",
"description": "MongoDB CRUD Integration Completed",
"status": "pending",
"completed": true,
"priority": "medium",
"createdAt": "2026-09-22T10:17:23.981Z",
"__v": 0
}
}
Response
Chart


POST
http://localhost:5000/api/tasks
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"description": "Testing validation error"
}
Status: 400 Bad Request
Size: 76 Bytes
Time: 20 ms
Response
Headers 7
Cookies
Results
Docs
{
"success": false,
"error": "Title is required and must be a non-empty string"
}
Response
Chart


GET
http://localhost:5000/api/tasks/507f1f77bcf86cd799439011
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"description": "Testing validation error"
}
Status: 404 Not Found
Size: 75 Bytes
Time: 20 ms
Response
Headers 7
Cookies
Results
Docs
Copy
{
"success": false,
"error": "Task with id 507f1f77bcf86cd799439011 not found"
}
Response
Chart


server.js
validateTaskId.js
errorHandler.js
Task.js
New Request
New Request
taskRoutes.js
BLACKBOX
GET
http://localhost:5000/api/tasks
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"title": "Prepare Practical File",
"description": "Complete practical documentation",
"priority": "low"
}
Status: 200 OK
Size: 670 Bytes
Time: 16 ms
Response
Headers 7
Cookies
Results
Docs
Copy
{
"success": true,
"count": 3,
"data": [
{
"_id": "6ab25833b24a7ea5d2fa5cb7",
"title": "Study MongoDB",
"description": "Practice Mongoose and CRUD",
"status": "pending",
"completed": false,
"priority": "medium",
"createdAt": "2026-09-22T10:28:03.402Z",
"__v": 0
},
{
"_id": "6ab25843b24a7ea5d2fa5cb8",
"title": "Complete DSA Practice",
"description": "Solve daily DSA questions",
Response
Chart





Conclusion:

This practical demonstrates the development of a RESTful backend server using Node.js, Express.js, MongoDB, and Mongoose for a Task Management system. It implements complete CRUD operations that allow users to create, retrieve, update, and delete tasks through REST API endpoints. The practical also demonstrates how to connect an Express application with MongoDB using Mongoose and define a structured task schema with validation, default values, and priority options. Appropriate HTTP methods and status codes are used to provide consistent and meaningful responses to clients. The API is tested using Thunder Client to verify the functionality and correctness of each endpoint, including successful operations and validation/error handling. Overall, this practical provides hands-on experience in building structured, reliable, and maintainable backend services using Node.js, Express.js, MongoDB, and Mongoose.